🎓 جزوه طلایی سیستم‌عامل

فصل ۱۰: حافظه مجازی - بخش هفتم و هشتم (فشرده‌سازی و حافظه هسته)

۱. پایان مبحث کوبیدگی (نکات تکمیلی)

الف) کنترل با فرکانس خطای صفحه (PFF)

راهکار مجموعه کاری (Working Set) کمی پیچیده است. راهکار ساده‌تر، کنترل مستقیم **نرخ خطای صفحه** است:

- **حد بالا (Upper Bound):** اگر نرخ خطا از این حد بیشتر شد ⬅ پردازه به قاب‌های بیشتری نیاز دارد (به آن قاب بدهید).
- **حد پایین (Lower Bound):** اگر نرخ خطا از این حد کمتر شد ⬅ پردازه قاب‌های زیادی اشغال کرده (از آن قاب بگیرید).
- **هدف:** نگه‌داشتن سیستم در یک وضعیت متعادل.

ب) تمرین فعلی (Current Practice)

در دنیای واقعی (از گوشی هوشمند تا سرور)، بهترین راه مقابله با کوبیدگی چیست؟

> **تدارک حافظه کافی:** سیستم را طوری طراحی کنید که رم فیزیکی کافی برای نگه داشتن مجموعه کاری تمام برنامه‌های مهم داشته باشد.

۲. فشرده‌سازی حافظه (Memory Compression)

یک جایگزین مدرن برای صفحه‌بندی (Swapping) سنتی، به ویژه در سیستم‌هایی که دیسک کند دارند یا فلش مموری محدود (مثل موبایل).

💡 ایده اصلی

به جای اینکه صفحات قربانی را در دیسک بنویسیم (که کند است)، آن‌ها را **فشرده** می‌کنیم و در بخشی از خودِ RAM نگه می‌داریم.

- **مثال:** فرض کنید ۳ قاب داده دارید. به جای نوشتن در دیسک، آن‌ها را فشرده کرده و در ۱ **قاب** جا می‌دهیم.
  - *نتیجه:* ۲ قاب آزاد می‌شود، بدون هیچ I/O دیسک!

📱 کاربرد در سیستم‌عامل‌ها

1. **موبایل (Android & iOS):** معمولاً از Swap به دیسک پشتیبانی نمی‌کنند (چون حافظه فلش محدودیت نوشتن دارد). فشرده‌سازی حافظه رکن اصلی مدیریت رم در موبایل است.
2. **دسکتاپ (Windows 10 & macOS):**
   - *macOS:* وقتی رم کم می‌آید، صفحات LRU را فشرده می‌کند. تست‌ها نشان داده این کار حتی از SSD سریع‌تر است.
   - *Windows 10:* از این تکنیک برای برنامه‌های UWP و سیستم استفاده می‌کند.

👨‍💻 چالش: ضریب فشرده‌سازی vs سرعت CPU

یک مسابقه بین "کاهش حجم" و "مصرف پردازنده" وجود دارد.

الگوریتم‌هایی مثل **WKdm** (اپل) و **Xpress** (مایکروسافت) استفاده می‌شوند که تعادل خوبی دارند (سرعت بالا و فشرده‌سازی ۳۰ تا ۵۰ درصد).

۳. تخصیص حافظه هسته (Kernel Memory Allocation)

آیا هسته (Kernel) هم مثل برنامه‌های کاربردی از لیست قاب‌های عادی استفاده می‌کند؟ **خیر!**

چرا؟

1. هسته به ساختمان داده‌هایی با **اندازه‌های متغیر** نیاز دارد (بسیار کوچکتر از یک صفحه ۴ کیلوبایتی). استفاده از صفحه‌بندی باعث "تکه‌تکه شدن داخلی" (Internal Fragmentation) شدید می‌شود.
2. برخی سخت‌افزارها نیاز دارند که حافظه فیزیکی **پیوسته (Contiguous)** باشد (برخلاف حافظه مجازی که پراکنده است).

برای حل این مشکل، دو روش اصلی وجود دارد:

الف) سیستم رفیق (Buddy System)

حافظه را از یک قطعه بزرگ با اندازه ثابت (Fixed-size Segment) تخصیص می‌دهد.

- **مکانیزم توان ۲:** درخواست‌ها را به نزدیک‌ترین توان ۲ گرد می‌کند (۱۶k۴، ۸k، k و...).
- **تقسیم (Splitting):** اگر یک درخواست ۲۱ کیلوبایتی بیاید و ما قطعه ۲۵۶ کیلوبایتی داشته باشیم:
  1. ۲۵۶ ⬅ ۱۲۸ و ۱۲۸
  2. ۱۲۸ ⬅ ۶۴ و ۶۴
  3. ۶۴ ⬅ ۳۲ و ۳۲
  4. یکی از ۳۲تایی‌ها را به درخواست ۲۱ کیلوبایتی می‌دهیم.
- **ائتلاف (Coalescing):** مزیت بزرگ این روش! وقتی دو نیمه (Buddy) آزاد می‌شوند، به سرعت با هم ترکیب شده و قطعه بزرگتر را می‌سازند.
- **عیب: تکه‌تکه شدن داخلی.** (مثلاً برای درخواست ۳۳ کیلوبایتی، باید ۶۴ کیلوبایت بدهیم ⬅ ۳۱ کیلوبایت هدر می‌رود).

ب) تخصیص قالبی (Slab Allocation) - روش برتر

این روش برای رفع مشکل تکه‌تکه شدن در سیستم Buddy طراحی شد.

- **قالب (Slab):** تشکیل شده از یک یا چند صفحه پیوسته فیزیکی.
- **کش (Cache):** برای هر نوع **شئ هسته** (Kernel Object)، یک کش مخصوص وجود دارد (مثلاً کشِ سمافورها، کشِ توصیف‌گر پردازه).
- **نحوه کار:**
  - کش‌ها با اشیای **پیش‌ساخته** پر می‌شوند.
  - وقتی هسته یک "سمافور" می‌خواهد، سیستم فقط یک شئ را از کش برداشته و "استفاده شده" علامت می‌زند.
- **وضعیت قالب‌ها:**
  1. **پر (Full):** تمام اشیا استفاده شده‌اند.
  2. **خالی (Empty):** همه اشیا آزادند.
  3. **نیمه‌پر (Partial):** مخلوط. (اولویت تخصیص با این‌هاست).

✅ مزایای Slab:

1. **هدررفت حافظه صفر:** چون دقیقاً به اندازه شئ حافظه می‌دهد (بدون تکه‌تکه شدن).
2. **سرعت بسیار بالا:** اشیا از قبل ایجاد شده‌اند و نیازی به مقداردهی اولیه و تخریب مکرر نیست (مناسب برای اشیایی که زیاد ساخته و پاک می‌شوند).

🐧 انواع در لینوکس:

- **SLAB:** پیاده‌سازی اصلی و اولیه.
- **SLOB:** نسخه سبک برای سیستم‌های تعبیه شده (Embedded) با حافظه کم.
- **SLUB:** جایگزین پیش‌فرض فعلی در لینوکس (سربار مدیریتی کمتر نسبت به SLAB).

📝 خلاصه طلایی بخش هفتم و هشتم:

1. **PFF:** کنترل کوبیدگی با تنظیم حد بالا و پایین برای نرخ خطا.
2. **Memory Compression:** فشرده‌سازی صفحات در RAM به جای Swap (حیاتی برای موبایل).
3. **Kernel Memory:** هسته نیازهای متفاوتی نسبت به کاربر دارد (اندازه متغیر + پیوستگی فیزیکی).
4. **Buddy System:** تقسیم حافظه به توان‌های ۲. (مشکل: پرتی حافظه / مزیت: ادغام سریع).
5. **Slab Allocation:** اختصاص کش‌های مخصوص برای اشیای هسته. (مزیت: سرعت بالا و بدون پرتی حافظه).